

ФГАОУ ВО «НИУ ИТМО»
ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ
И КОМПЬЮТЕРНОЙ ТЕХНИКИ

Лабораторная работа №5

Решение дифференциальных уравнений

(по дисциплине «Вычислительная математика»)

Выполнил:

Лабин Макар Андреевич,
ВЫЧМ 3.2, Р3231, 466449.

Проверила:

Перл Ольга Вячеславовна

ИТМО

г. Санкт-Петербург, Россия
2026

Содержание

1	Постановка задач	1
2	Выполнение работы	2
2.1	Описание метода	2
2.1.1	Постановка задачи Коши для ОДУ	2
2.1.2	Метод Эйлера	2
2.1.3	Критерий останова	3
2.2	Блок-схема численного метода	3
2.3	Листинг численного метода	6
2.4	Результаты работы программы	11
2.4.1	Подготовка дифференциальных уравнений	11
2.4.2	Графики полученных решений	12
3	Заключение	19
4	Источники литературы	21

1 Постановка задач

Дано обыкновенное дифференциальное уравнение первого порядка с начальным условием вида:

$$y' = f(x, y), \quad y(a) = y_0$$

Необходимо найти приближённое решение задачи Коши на интервале $[a, b]$ с заданным параметром точности ε — максимальной допустимой разницей между значениями $y(b)$, которые получены при последовательном уточнении шага интегрирования.

В варианте №1 вычисление осуществляется методом Эйлера. При реализации алгоритма необходимо учитывать следующие условия:

- правая часть уравнения $f(x, y)$ выбирается по номеру из набора подготовленных дифференциальных уравнений;
- шаг интегрирования h должен определяться и автоматически пересчитываться до тех пор, пока не будет выполнено условие точности;
- на графиках должны быть отображены начальное условие (a, y_0) , численное решение, полученное методом Эйлера, и аналитическое решение (при его наличии) для визуального сравнения и оценки накопления погрешности;
- в ходе анализа необходимо исследовать устойчивость метода, а также рассмотреть сценарии практического применения метода.

То есть в ходе лабораторной работы будут выполнены следующие задачи:

1. Разработка итерационного алгоритма решения задачи Коши методом Эйлера с автоматическим подбором шага h и контролем точности по заданному параметру ε .
2. Реализация механизма выбора функции $f(x, y)$ по индексу, построения графиков численного и аналитического решений, а также визуализации начальных условий.
3. Организация ввода данных в формате $a \ b \ y_0 \ f_n \ \varepsilon$, формирование вывода в виде приближённого значения $y(b)$ и проведение анализа устойчивости и погрешности полученного решения.

Теоретический раздел подготовлен с опорой на внешний источник[1].

2 Выполнение работы

2.1 Описание метода

2.1.1 Постановка задачи Коши для ОДУ

Обыкновенным дифференциальным уравнением называется уравнение, которое связывает независимую переменную x , искомую функцию $y(x)$ и её производные различных порядков.

Если в уравнение входит только первая производная, оно называется **ОДУ первого порядка**:

$$y' = f(x, y).$$

Задачей Коши (или *задачей с начальными условиями*) для дифференциального уравнения первого порядка называется задача поиска функции $y(x)$, которая удовлетворяет самому уравнению на интервале $[a, b]$, а также заданному *начальному условию*:

$$y(a) = y_0,$$

где y_0 — известное начальное значение в точке $x = a$.

2.1.2 Метод Эйлера

Рассмотрим простейший одношаговый численный метод — **метод Эйлера**. Пусть на отрезке $[a, b]$ задан некоторый постоянный шаг $h = (b - a)/n$, тогда с его помощью определим узлы сетки:

$$x_i = a + ih, \quad i = \overline{1, n}$$

Разложим истинное решение $y(x)$ в окрестности точки x_i в ряд Тейлора:

$$y(x_{i+1}) = y(x_i + h) = y(x_i) + hy'(x_i) + \frac{h^2}{2}y''(\xi_i), \quad \xi_i \in (x_i, x_{i+1}).$$

Метод Эйлера строит строит точки $\omega_i \approx y(x_i)$, отбрасывая члены высших порядков малости. Учитывая, что $y'(x_i) = f(x_i, y(x_i))$, имеем:

$$\begin{aligned} \omega_0 &= y_0 \\ \omega_{i+1} &= \omega_i + hf(x_i, \omega_i), \quad i = \overline{0, n-1} \end{aligned}$$

Для поиска *локальной погрешности* метода предположим, что в узле x_i вычисленное значение совпадает с точным, то есть $\omega_i = y(x_i)$. Разложим истинное решение $y(x_{i+1})$ в окрестности точки x_i в ряд Тейлора:

$$y(x_{i+1}) = y(x_i) + h \cdot y'(x_i) + \frac{h^2}{2} \cdot y''(\xi_i), \quad \xi_i \in (x_i, x_{i+1}).$$

По определению ОДУ и методу Эйлера:

$$y'(x_i) = f(x_i, y(x_i)) \implies |y(x_{i+1}) - \omega_{i+1}| = \frac{h^2}{2}|y''(\xi_i)| = O(h^2) \quad \square$$

Чтобы найти *глобальную погрешность*, учтём, что при работе на всем отрезке $[a, b]$ ошибки на каждом шаге суммируются. Чтобы пройти весь интервал, алгоритму требуется совершить n шагов.

В худшем случае, если локальные погрешности накапливаются с одинаковым знаком, полная накопленная ошибка пропорциональна произведению числа шагов на локальную ошибку одного шага:

$$|y(b) - y_n| \approx n \cdot O(h^2) = \frac{b-a}{h} \cdot O(h^2) = (b-a) \cdot O(h) = O(h) \quad \square$$

Таким образом, метод Эйлера обладает **первым порядком точности**.

2.1.3 Критерий останова

Чтобы достичь погрешности решения не более чем ε , воспользуемся следующим наблюдением[2]. Метод Эйлера имеет первый порядок точности. По определению это означает, что приближенное решение $y_h(b)$, полученное с шагом h , связано с точным решением $y(b)$ соотношением:

$$y(b) = y_h(b) + C \cdot h + O(h^2),$$

где C — константа, которая не зависит от h .

Если мы уменьшим шаг интегрирования в два раза и найдем решение $y_{h/2}(b)$, то для него справедливо:

$$y(b) = y_{h/2}(b) + C \cdot \frac{h}{2} + O(h^2)$$

Вычтем из второго равенства первое, получив следующее соотношение:

$$|y_{h/2}(b) - y_h(b)| \approx \left| C \cdot \frac{h}{2} \right|$$

Нехитрой подстановкой во второе равенство третьего заключим:

$$|y(b) - y_{h/2}(b)| \approx |y_{h/2}(b) - y_h(b)|$$

Таким образом, для достижения точности ε алгоритм должен итерационно уменьшать шаг h до тех пор, пока не выполнится критерий останова:

$$|y_{h/2}(b) - y_h(b)| \leq \varepsilon$$

2.2 Блок-схема численного метода

На вход реализуемого метода подаются следующие данные:

- f — номер дифференциального уравнения; целое число.
- ε — требуемая точность вычислений, то есть максимально допустимая разность между двумя итерациями; вещественное число двойной точности.
- a, b — границы интервала вычислений; вещественные числа двойной точности.
- y_a — начальное условие в точке a ; вещественное число двойной точности.

- M — максимально допустимое количество разбиений сетки; целое число.

На выходе метода ожидается вещественное число двойной точности — приближённое значение $y(b)$.

Остановка итерационного процесса происходит при достижении заданной точности ε или при достижении лимита разбиений M . Шаг сетки на каждой итерации уменьшается вдвое за счёт удвоения количества узлов n .

Далее приводятся блок-схемы, которые реализуют вспомогательный алгоритм расчёта по сетке и основной численный метод.

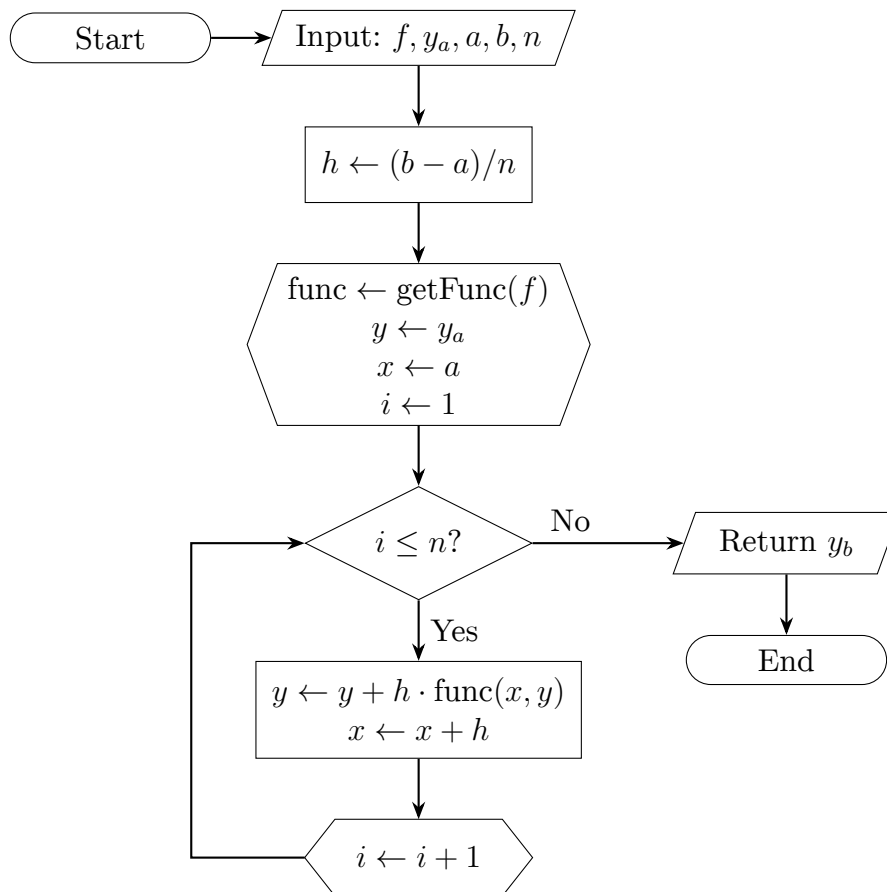


Схема 1 — Блок-схема функции вычисления по равномерной сетке.

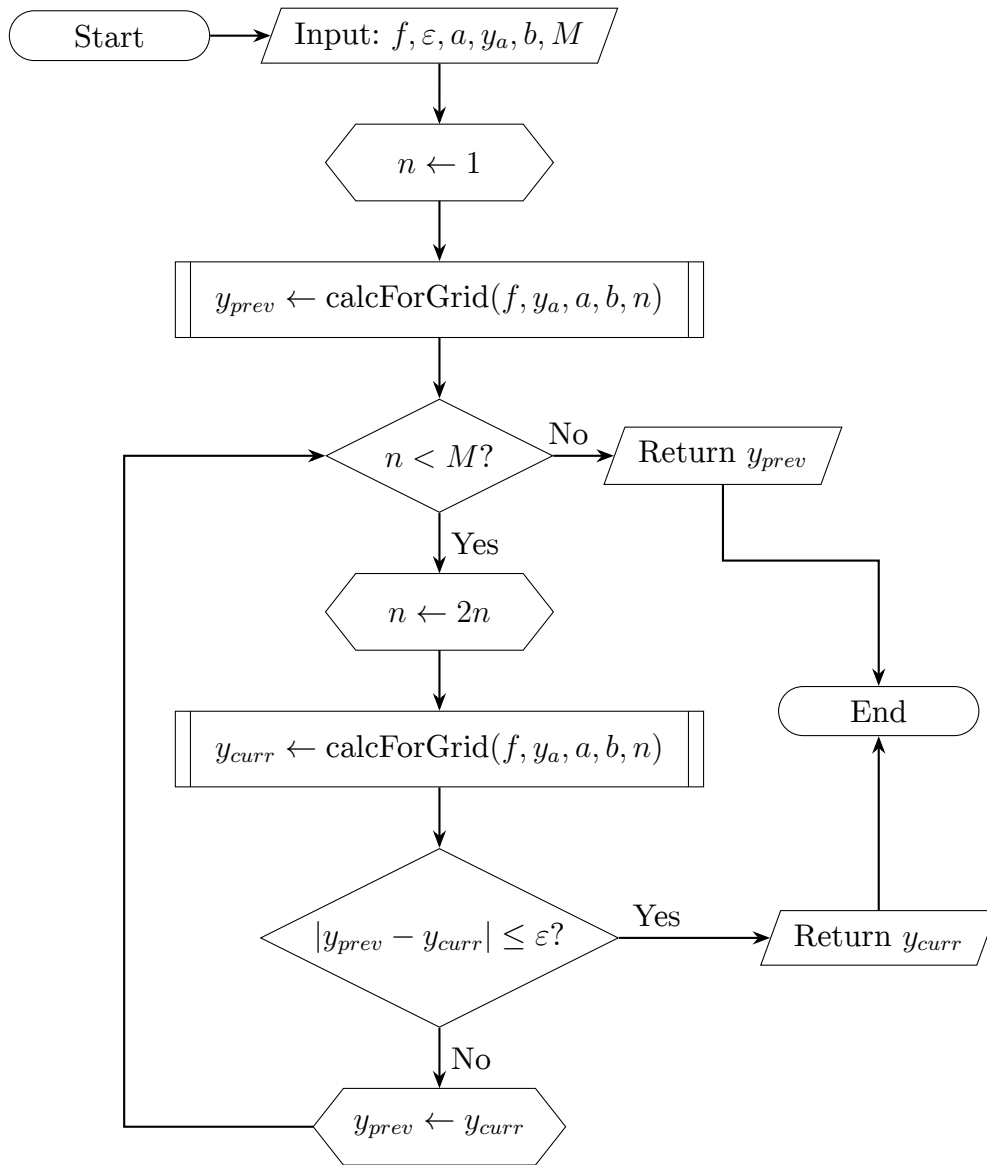


Схема 2 — Блок-схема основного алгоритма решения методом Эйлера с автоматическим выбором шага.

2.3 Листинг численного метода

Для реализации блок-схем воспользуемся языком программирования C++.

```
1 #ifndef DIFFERENTIAL_H
2 #define DIFFERENTIAL_H
3
4 #include "preamble.h"
5 #include <cstdlib>
6
7 std::size_t ITERATIONS_MAX = 1e6;
8
9 struct EulerSolution {
10     double answer;
11     std::size_t n_grid;
12 };
13
14 double calcForGrid(int f, double y_a, double a, double b, std::
15     size_t n_grid) {
16     double h = (b - a) / n_grid;
17     std::function<double(double,double)> func = get_function(f);
18     double y_b = y_a;
19     double x = a;
20     for (std::size_t i = 1; i <= n_grid; i++) {
21         y_b += h * func(x, y_b);
22         x += h;
23     }
24     return y_b;
25 }
26 EulerSolution solveByEuler(int f, double epsilon, double a,
27     double y_a, double b) {
28     std::size_t n_grid = 2;
29     double y_b_h = calcForGrid(f, y_a, a, b, n_grid);
30     while (n_grid < ITERATIONS_MAX) {
31         n_grid *= 2;
32         double y_b_h_2 = calcForGrid(f, y_a, a, b, n_grid);
33         if (std::abs(y_b_h - y_b_h_2) <= epsilon) {
34             return { y_b_h_2, n_grid };
35         }
36         y_b_h = y_b_h_2;
37     }
38     return { y_b_h, n_grid };
39 }
40 #endif
```

Листинг 1 — Файл differential.hpp

```
1 #ifndef INTERPOLATE_H
2 #define INTERPOLATE_H
3
4 #include <vector>
```

```

5 #include <tuple>
6 #include <functional>
7 #include <algorithm>
8 #include <cmath>
9
10 struct InterpolationResult {
11     std::vector<std::tuple<double, double>> points;
12     std::size_t chebyshev_k;
13 };
14
15 std::vector<double> GetChebyshevRoots(double a, double b, std::
    size_t n_size) {
16     std::vector<double> roots(n_size);
17     const double pi = std::acos(-1.0);
18
19     for (std::size_t i = 0; i < n_size; i++) {
20         double x_standard = std::cos((2.0 * i + 1.0) / (2.0 * n_size
            ) * pi);
21         roots[i] = 0.5 * (a + b) + 0.5 * (b - a) * x_standard;
22     }
23
24     return roots;
25 }
26
27 InterpolationResult InterpolateLagrange(double a, double b, std
    ::function<double(double)> func, double eps, std::size_t
    n_lagrange_max, std::size_t dots_num) {
28     std::vector<std::tuple<double, double>> lagrange_values(
        dots_num + 1);
29     std::vector<double> prev_values(dots_num + 1, 0.0);
30     std::vector<double> curr_values(dots_num + 1, 0.0);
31
32     auto eval_lagrange = [](double x, const std::vector<double>&
        nodes, const std::vector<double>& values) {
33         double total_sum = 0.0;
34         std::size_t n = nodes.size();
35         for (std::size_t i = 0; i < n; ++i) {
36             double term = values[i];
37             for (std::size_t j = 0; j < n; ++j) {
38                 if (j != i) {
39                     term *= (x - nodes[j]) / (nodes[i] - nodes[j]);
40                 }
41             }
42             total_sum += term;
43         }
44         return total_sum;
45     };
46
47     std::size_t final_k = 0;
48     for (std::size_t k = 5; k <= n_lagrange_max; ++k) {
49         final_k = k;

```

```

50     auto nodes = GetChebyshevRoots(a, b, k);
51     std::vector<double> values(k);
52     for (std::size_t j = 0; j < k; ++j) {
53         values[j] = func(nodes[j]);
54     }
55
56     for (std::size_t i = 0; i <= dots_num; ++i) {
57         double x = (i == 0) ? a : ((i == dots_num) ? b : a + i * (
58             b - a) / dots_num);
59         curr_values[i] = eval_lagrange(x, nodes, values);
60     }
61
62     if (k > 1) {
63         double max_diff = 0.0;
64         for (std::size_t i = 0; i <= dots_num; ++i) {
65             max_diff = std::max(max_diff, std::abs(curr_values[i] -
66                 prev_values[i]));
67         }
68         if (max_diff <= eps) {
69             break;
70         }
71     }
72     prev_values = curr_values;
73
74     for (std::size_t i = 0; i <= dots_num; ++i) {
75         double x = (i == 0) ? a : ((i == dots_num) ? b : a + i * (b
76             - a) / dots_num);
77         lagrange_values[i] = std::make_tuple(x, curr_values[i]);
78     }
79
80     return {lagrange_values, final_k};
81 }
82 #endif

```

Листинг 2 — Файл interpolate.hpp

```

1  #include <iostream>
2  #include <vector>
3  #include <cmath>
4  #include <iomanip>
5  #include <fstream>
6  #include <string>
7  #include <filesystem>
8
9  #include "lib/differential.hpp"
10 #include "lib/interpolate.hpp"
11
12 struct TestCase {
13     int func_number;
14     int case_index;

```

```

15 double a;
16 double y_a;
17 double b;
18 double epsilon;
19 };
20
21 double getExactResult(int f, double a, double y_a, double b) {
22     switch (f) {
23         case 1:
24             return y_a + std::cos(a) - std::cos(b);
25         case 2:
26             return y_a * std::exp((b * b - a * a) / 4.0);
27         case 3: {
28             double C = (y_a * y_a - 2.0 * a - 1.0) * std::exp(-2.0 * a
29             );
30             double expr = 2.0 * b + 1.0 + C * std::exp(2.0 * b);
31             return (expr >= 0.0) ? std::sqrt(expr) : std::nan("");
32         }
33         case 4:
34             return (y_a + a + 1.0) * std::exp(b - a) - b - 1.0;
35         case 5: {
36             double denom = (1.0 / y_a) - (b - a);
37             return (std::abs(denom) > 1e-9) ? (1.0 / denom) : std::nan(
38             "");
39         }
40         default:
41             return 0.0;
42     }
43 }
44
45 void GenerateTrajectoryData(const TestCase& t) {
46     namespace fs = std::filesystem;
47
48     fs::path dir_path = "output";
49     fs::create_directories(dir_path);
50
51     std::string prefix = "f" + std::to_string(t.func_number) + "
52     _case" + std::to_string(t.case_index);
53     fs::path euler_path = dir_path / (prefix + "_euler.txt");
54     fs::path exact_path = dir_path / (prefix + "_exact.txt");
55
56     std::ofstream euler_file(euler_path);
57     std::ofstream exact_file(exact_path);
58
59     if (!euler_file.is_open() || !exact_file.is_open()) return;
60
61     std::size_t samples = 100;
62     std::size_t euler_iterations = 0;
63
64     auto eval_euler = [t, &euler_iterations](double x) {
65         EulerSolution solution = solveByEuler(t.func_number, t.

```

```

        epsilon, t.a, t.y_a, x);
63     euler_iterations = std::max(euler_iterations, solution.
        n_grid);
64     return solution.answer;
65 };
66 auto eval_exact = [t](double x) {
67     return getExactResult(t.func_number, t.a, t.y_a, x);
68 };
69
70 InterpolationResult euler_result = InterpolateLagrange(t.a, t.
    b, eval_euler, t.epsilon, samples, samples);
71 InterpolationResult exact_result = InterpolateLagrange(t.a, t.
    b, eval_exact, t.epsilon, samples, samples);
72 euler_file << euler_result.chebyshev_k << " " << t.a << " " <<
    t.b << " " << t.epsilon << " " << t.y_a << " " <<
    euler_iterations << '\n';
73 exact_file << exact_result.chebyshev_k << '\n';
74 for (std::tuple<double, double> point : euler_result.points) {
75     euler_file << std::get<0>(point) << " " << std::get<1>(point
        ) << "\n";
76 }
77 for (std::tuple<double, double> point : exact_result.points) {
78     exact_file << std::get<0>(point) << " " << std::get<1>(point
        ) << "\n";
79 }
80 }
81
82 int main() {
83     std::cout << std::fixed << std::setprecision(6);
84
85     std::vector<TestCase> tests = {
86         {1, 1, 0.0, 0.0, 3.141592, 1e-4},
87         {1, 2, 0.0, 1.0, 6.283185, 1e-4},
88         {2, 1, 0.0, 1.0, 2.0, 1e-4},
89         {2, 2, 0.0, 0.2, 7.2, 1e-4},
90         {3, 1, 0.0, 2.0, 1.5, 1e-4},
91         {3, 2, 0.0, 1.0, 0.8, 1e-4},
92         {4, 1, 0.0, 0.0, 1.0, 1e-4},
93         {4, 2, 0.0, -1.0, 2.0, 1e-4},
94         {5, 1, 0.0, 0.5, 1.5, 1e-4},
95         {5, 2, 0.0, 0.8, 1.1, 1e-4},
96     };
97
98     for (const TestCase& t : tests) {
99         GenerateTrajectoryData(t);
100         std::cout << "Trajectory data written to disk.\n";
101     }
102
103     return 0;
104 }

```

Листинг 3 — Файл main.cpp

2.4 Результаты работы программы

2.4.1 Подготовка дифференциальных уравнений

Рассмотрим пять обыкновенных дифференциальных уравнений первого порядка вида $y' = f(x, y)$ с разными аналитическими особенностями для проверки метода Эйлера:

1. *Функция, которая зависит лишь от независимой переменной:*

$$f_1(x, y) = \sin x.$$

Аналитическое решение:

$$y(x) = y_a + \cos a - \cos x$$

Задача фактически сводится к нахождению первообразной — используем для проверки корректности шага.

2. *Линейное разделяющееся уравнение:*

$$f_2(x, y) = \frac{x \cdot y}{2}.$$

Аналитическое решение:

$$y(x) = y_a \cdot \exp\left(\frac{x^2 - a^2}{4}\right)$$

Траектория растёт экспоненциально при $x > 0$, что позволяет оценить накопление погрешности метода.

3. *Нелинейное уравнение:*

$$f_3(x, y) = y - \frac{2x}{y}.$$

Аналитическое решение:

$$y(x) = \sqrt{2x + 1 + Ce^{2x}}$$

где константа C определяется через начальные условия:

$$C = (y_a^2 - 2a - 1)e^{-2a}$$

Если в процессе интегрирования численное решение приближается к нулю или выражение под корнем в аналитическом решении становится отрицательным, траектория разрушается (*деление на ноль или выход за область определения*).

4. *Линейное неоднородное уравнение:*

$$f_4(x, y) = x + y.$$

Аналитическое решение:

$$y(x) = (y_a + a + 1)e^{x-a} - x - 1$$

Решение содержит как линейную, так и экспоненциальную части.

5. Нелинейное уравнение со слабой устойчивостью:

$$f_5(x, y) = y^2.$$

Аналитическое решение:

$$y(x) = \frac{1}{(1/y_a) - (x - a)}$$

При некоторых начальных условиях знаменатель может обратиться в ноль, и решение уходит в бесконечность.

2.4.2 Графики полученных решений

В лабораторной работе №1 мне был выдан вариант с *интерполяционным многочленом в форме Лагранжа*, который использует интерполяционные узлы от *многочлена Чебышёва*. Поэтому графики далее будут использовать указанный метод для отображения аналитического и численного решений дифференциального уравнения.

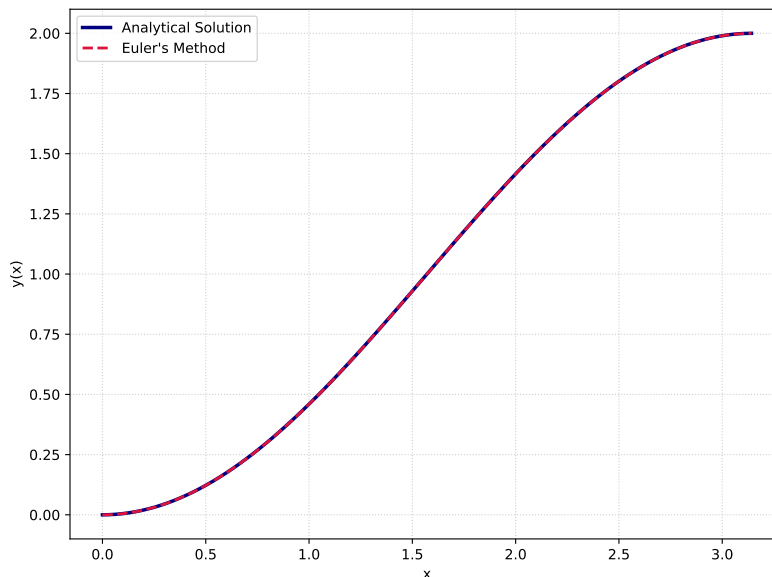


Рис. 1 — Графическое сравнение решений ОДУ №1.

Параметры графика на рис. 1:

- начальное условие — $y(0) = 0$;
- интересующая точка — 3.14159;
- малая константа — 0.0001;
- число узлов в методе Эйлера — 16384;
- число узлов Чебышёва для численного решения — 7;
- число узлов Чебышёва для аналитического решения — 7.

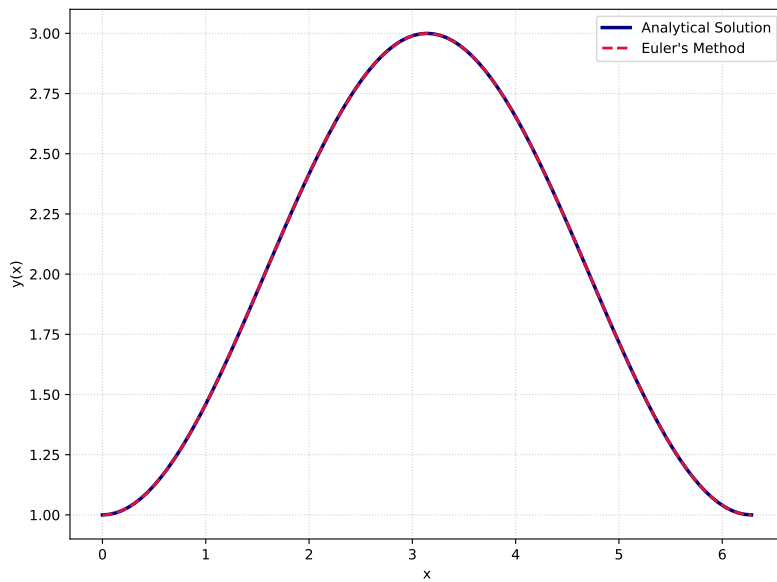


Рис. 2 — Графическое сравнение решений ОДУ №1.

Параметры графика на рис. 2:

- начальное условие — $y(0) = 1$;
- интересующая точка — 6.28318;
- малая константа — 0.0001;
- число узлов в методе Эйлера — 32768;
- число узлов Чебышёва для численного решения — 10;
- число узлов Чебышёва для аналитического решения — 10.

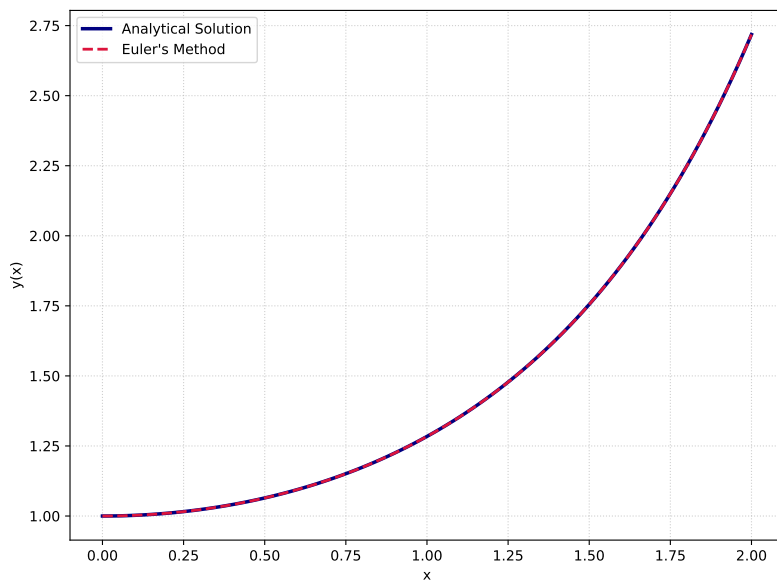


Рис. 3 — Графическое сравнение решений ОДУ №2.

Параметры графика на рис. 3:

- начальное условие — $y(0) = 1$;
- интересующая точка — 2;
- малая константа — 0.0001;
- число узлов в методе Эйлера — 65536;
- число узлов Чебышёва для численного решения — 8;
- число узлов Чебышёва для аналитического решения — 8.

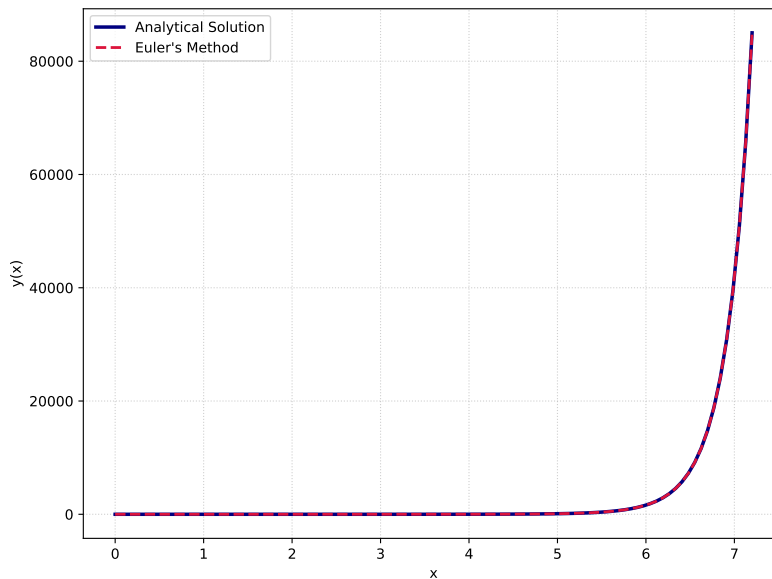


Рис. 4 — Графическое сравнение решений ОДУ №2.

Параметры графика на рис. 4:

- начальное условие — $y(0) = 0.2$;
- интересующая точка — 7.2;
- малая константа — 0.0001;
- число узлов в методе Эйлера — 1048576;
- число узлов Чебышёва для численного решения — 28;
- число узлов Чебышёва для аналитического решения — 28.

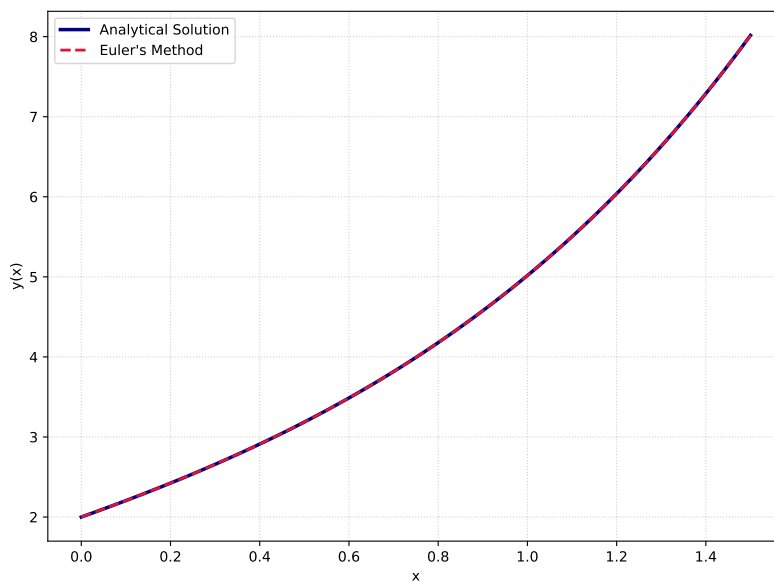


Рис. 5 — Графическое сравнение решений ОДУ №3.

Параметры графика на рис. 5:

- начальное условие — $y(0) = 2$;
- интересующая точка — 1.5;
- малая константа — 0.0001;
- число узлов в методе Эйлера — 131072;
- число узлов Чебышёва для численного решения — 7;
- число узлов Чебышёва для аналитического решения — 7.

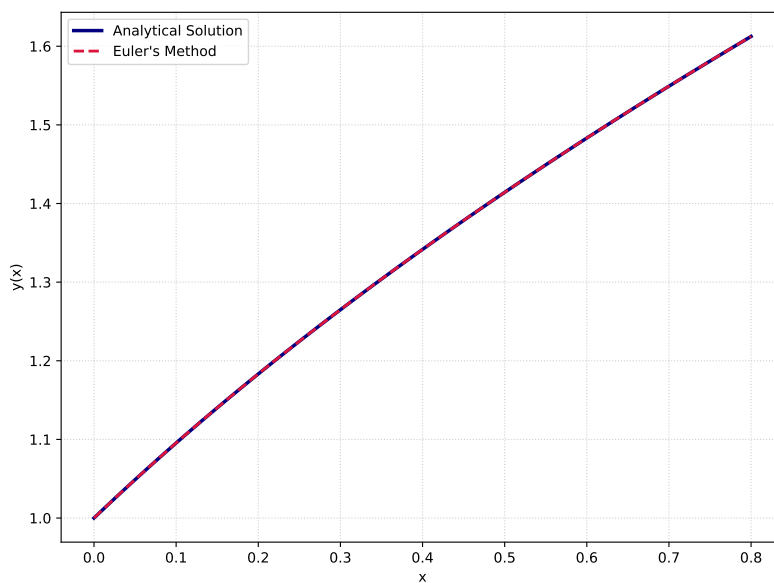


Рис. 6 — Графическое сравнение решений ОДУ №3.

Параметры графика на рис. 6:

- начальное условие — $y(0) = 1$;
- интересующая точка — 0.8;
- малая константа — 0.0001;
- число узлов в методе Эйлера — 4096;
- число узлов Чебышёва для численного решения — 6;
- число узлов Чебышёва для аналитического решения — 6.

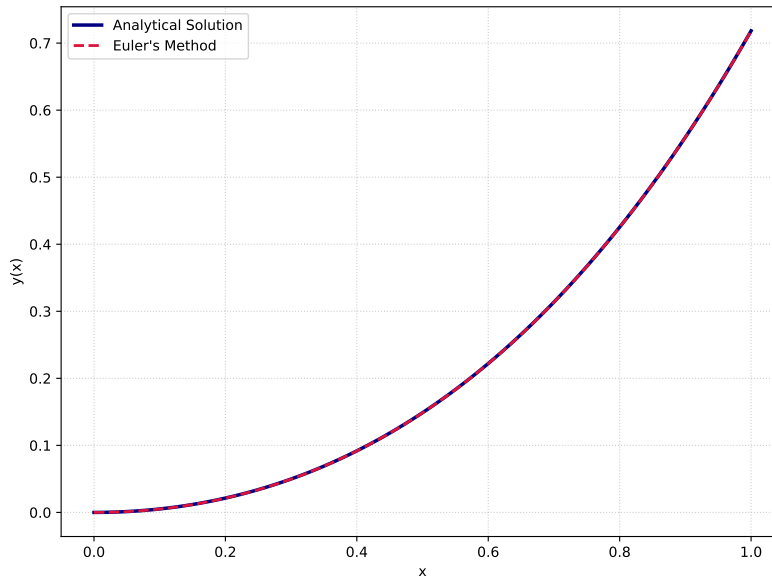


Рис. 7 — Графическое сравнение решений ОДУ №4.

Параметры графика на рис. 7:

- начальное условие — $y(0) = 0$;
- интересующая точка — 1;
- малая константа — 0.0001;
- число узлов в методе Эйлера — 16384;
- число узлов Чебышёва для численного решения — 6;
- число узлов Чебышёва для аналитического решения — 6.

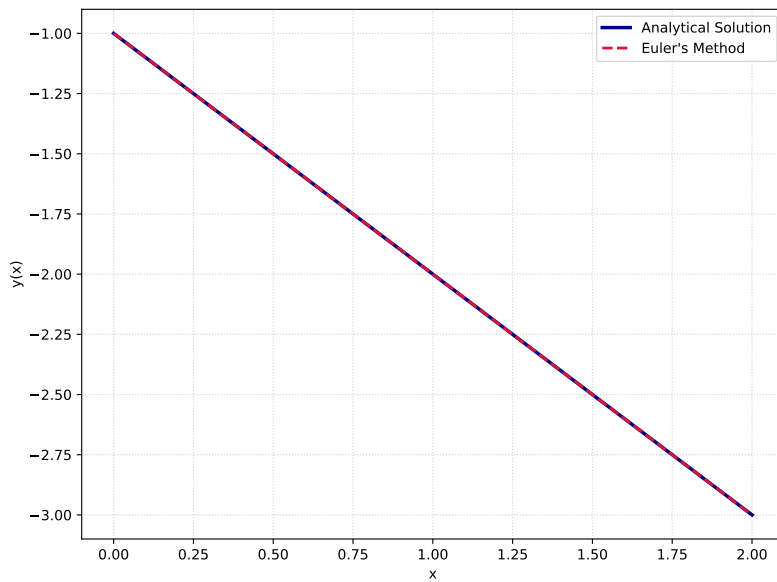


Рис. 8 — Графическое сравнение решений ОДУ №4.

Параметры графика на рис. 8:

- начальное условие — $y(0) = -1$;
- интересующая точка — 2;
- малая константа — 0.0001;
- число узлов в методе Эйлера — 4;
- число узлов Чебышёва для численного решения — 6;
- число узлов Чебышёва для аналитического решения — 6.

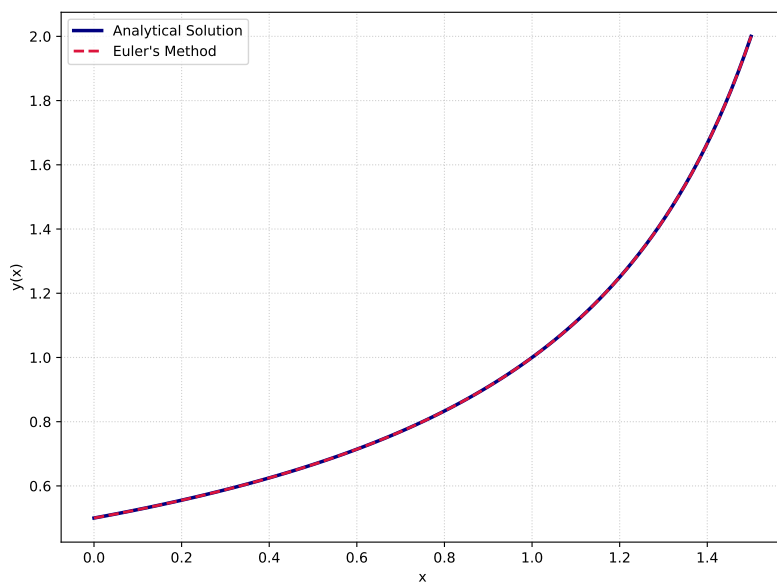


Рис. 9 — Графическое сравнение решений ОДУ №5.

Параметры графика на рис. 9:

- начальное условие — $y(0) = 0.5$;
- интересующая точка — 1.5;
- малая константа — 0.0001;
- число узлов в методе Эйлера — 131072;
- число узлов Чебышёва для численного решения — 11;
- число узлов Чебышёва для аналитического решения — 11.

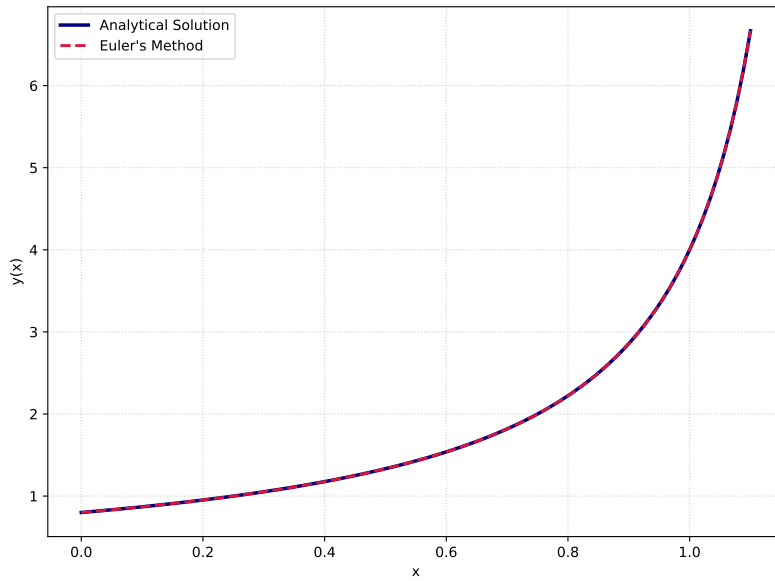


Рис. 10 — Графическое сравнение решений ОДУ №5.

Параметры графика на рис. 10:

- начальное условие — $y(0) = 0.8$;
- интересующая точка — 1.1;
- малая константа — 0.0001;
- число узлов в методе Эйлера — 1048576;
- число узлов Чебышёва для численного решения — 17;
- число узлов Чебышёва для аналитического решения — 17.

3 Заключение

После выполнения лабораторной работы у меня появились ответы на следующие вопросы.

Почему метод Эйлера везде совпал с аналитическим решением? Подозреваю, что это связано с адаптивным шагом (*зависимости от ε*) и, как следствие, большим числом узлов. В частности, некоторые примеры изображены с предварительным вычислением значений на миллионе узлов (*хотя мы отрисовываем лишь 100 из них для интерполяционного многочлена*).

Какая асимптотическая сложность у алгоритма? Расчёт метода Эйлера на фиксированной сетке из N шагов имеет линейную сложность $O(N)$, так как алгоритм один раз последовательно проходит по всем узлам интервала.

В адаптивной реализации для проверки точности по правилу Рунге сетка каждый раз увеличивается ровно в два раза:

$$2 + 4 + 8 + \dots + N_{max} = 2N_{max} - 2 = O(N_{max})$$

где N_{max} — финальное число узлов сетки, на котором разность двух приближений стала меньше ϵ .

То есть итоговая асимптотическая сложность алгоритма адаптивного метода Эйлера линейна и составляет $O(N_{max})$.

В чём преимущество метода Эйлера? Алгоритм невероятно прост: что его блок-схема проста, что программная реализация (*визуализация не в счёт*). Также расчёт каждого шага занимает крайне малое время — требуется всего одно вычисление правой части ОДУ. Кроме того, явный метод Эйлера не требует хранения предыстории вычислений в памяти.

Какие недостатки обнаружены у метода Эйлера? Метод имеет первый порядок точности $O(h)$. Из-за этого для достижения высокой точности приходится брать очень-очень малый шаг, что кратно увеличивает число итераций. По идее ещё должна быстро копиться ошибка, но на десяти примерах этого я не заметил. Но однозначно могу отметить, что при отдалении интересующей точки время расчёта существенно увеличивается.

Когда метод не применим В теории метод Эйлера неприменим для решения жёстких систем дифференциальных уравнений, где ограничение на шаг оказывается слишком жёстким для сохранения устойчивости. Однако у меня не возникло проблем для такой системы (*см. №5*).

Является ли метод устойчивым? Для начала нужно понять, что такое устойчивый метод.

Устойчивость численного метода — свойство ограничивать рост погрешностей в процессе итерационного вычисления.

Если метод неустойчив, то случайная ошибка на начальных итерациях будет постепенно увеличиваться, что приведет к расходимости численного решения с аналитическим.

Метод Эйлера является *условно устойчивым*, то есть она зависит от величины шага h . И если выбрать h больше некоторого критического, то накопление ошибки приведет к отклонению графика решения от аналитического.

4 Источники литературы

1. *Burden R. L., Faires J. D.* Numerical Analysis. — 9th. — Boston, MA : Brooks/Cole, Cengage Learning, 2011. — ISBN 978-0-538-73351-9. — URL: <https://www.cengage.com/>.
2. Правило Рунге / Википедия. — URL: https://ru.wikipedia.org/wiki/%D0%9F%D1%80%D0%B0%D0%B2%D0%B8%D0%BB%D0%BE_%D0%A0%D1%83%D0%BD%D0%B3%D0%B5 (дата обр. 23.05.2026).